Vietnam National University

University of Science

Faculty Information Technology

Computer Science

Project 1 - Socket Programming

Subject: CS494 - Internetworking Protocol

Student:

Doan Cong Pho - 23125089

Le Kim Chau - 23125078

Instructor:

Dr. Tran Trung Dung

Nguyen Thanh Quan, MSc

Chung Thuy Linh, MSc

Ngày 26 tháng 7 năm 2026

Mục lục

1 Application Scenario & Protocol Interaction

1.1 Overview

The Hybrid FTP application decouples the control plane from the data plane, mirroring RFC 959's architectural philosophy while adapting it to a connectionless UDP data channel:

- **Control channel** – TCP, port 2121. One command/reply per line, CRLF-terminated, three-digit FTP-style reply codes (Section 2.3 of the project spec).
- **Data channel** – UDP. File payload and (Advanced Level) directory listing text, framed with a 9-byte custom header (Section 2 of this report). Because UDP provides no delivery guarantee, a from-scratch reliability sub-protocol (Go-Back-N, Excellent Level) is layered on top, opt-in via `config.ini`.

Three data-channel operating modes are supported (Advanced Level): **FIXED** (Basic Level default – a single, server-wide UDP socket, address learned via one `HELLO` datagram), **PASSIVE** (`PASV` – server opens a per-session ephemeral socket and announces it), and **ACTIVE** (`PORT` – client announces its own address; the server still opens a per-session socket and reports its own port back, a deliberate deviation from RFC 959 discussed in Section 3.4).

1.2 Sequence Diagrams

Figure 1 traces the complete lifecycle of one session: TCP handshake, authentication, the FIXED-mode UDP address-learning handshake, an upload (`STOR`) using the Go-Back-N reliable-UDP layer – including one simulated packet loss and its recovery – and end-to-end hash verification.

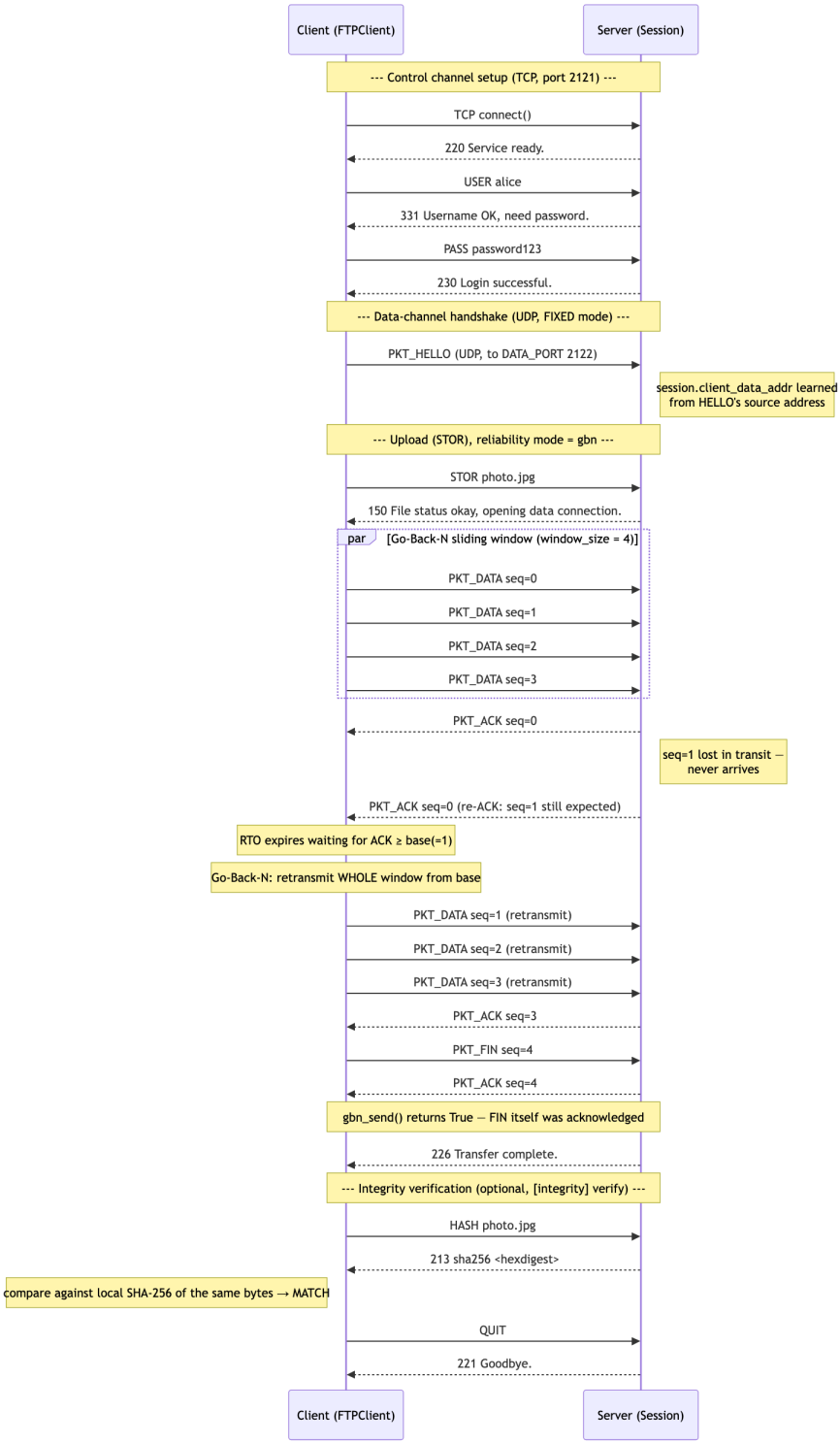

Figure 1: Full TCP + UDP session lifecycle, including a Go-Back-N retransmission after a simulated packet loss on `seq=1`.

Figure 2 details the two Advanced Level mode-negotiation paths side by side.

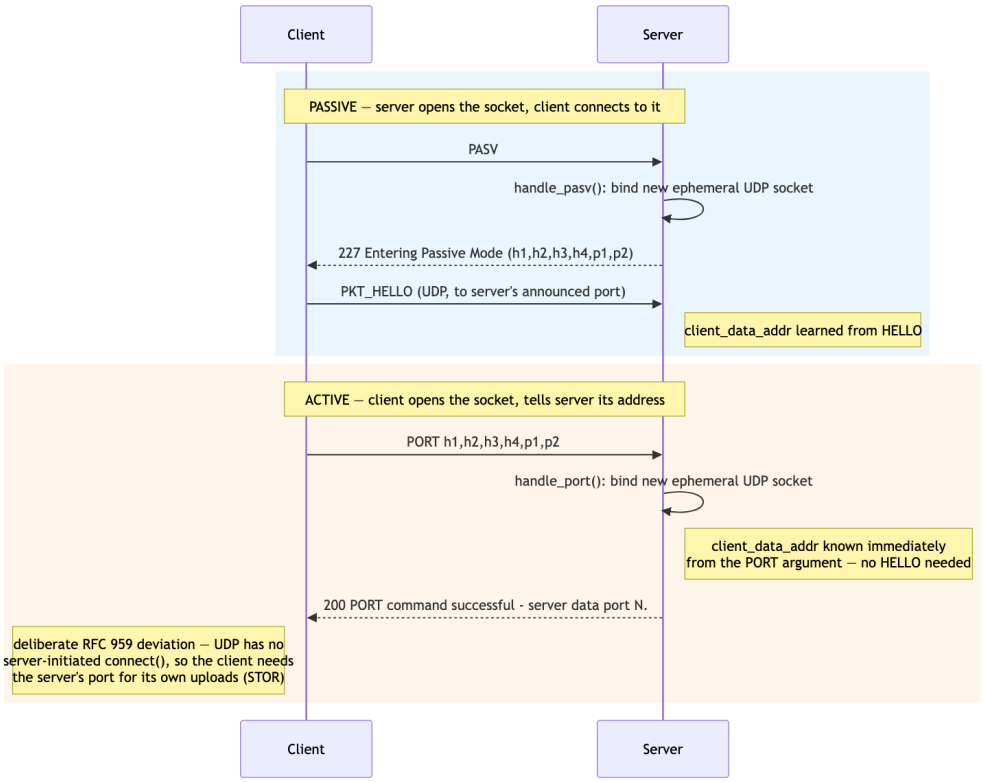

Figure 2: Passive (`PASV`) vs. Active (`PORT`) mode negotiation.

2 Project-Wide Data Structures

2.1 TCP Control-Channel Line Protocol

The control channel carries no binary framing – every message is a single UTF-8/ASCII line terminated by `"\r\n"` (CRLF), matching RFC 959. Both directions use it: client → server commands (e.g. `STOR photo.jpg`) and server → client replies (e.g. `226 Transfer complete.`). Table 1 summarizes the reply code categories in use (Section 2.3 of the project spec).

Table 1: Server reply code categories in use

Code range	**Examples used in this project**
1xx	`150` File status okay, opening data connection.
2xx	`200` Command OK; `220` Service ready; `226` Transfer complete; `230` Login successful; `250` Directory changed.
3xx	`331` Username OK, need password.
4xx	`425` Can't open data connection; `426` Connection closed, transfer aborted.
5xx	`500`/`501` Syntax error; `502` Not implemented; `530` Not logged in; `550` File unavailable.

2.2 UDP Custom Packet Header

Every UDP datagram on the data channel is framed identically, regardless of packet type or reliability mode (`common.py`, `HEADER_FMT = "!BII"`, network byte order):

Table 2: UDP packet header layout – 9 bytes, followed by up to `CHUNK_SIZE` (1024) bytes of payload

Offset	**Field**	**Size / Type**
0	`type`	1 byte, unsigned (`B`)
1	`seq`	4 bytes, unsigned (`I`)
5	`checksum`	4 bytes, unsigned (`I`) – CRC32 of `payload`
9...	`payload`	0–1024 bytes

Note that `PKT_ACK` reuses the exact same 9-byte header – no wire-format change was needed to add reliability; only a new `type` value and an interpretation convention for `seq`.

Table 3: Packet `type` values

Value	Name	Meaning
0	`PKT_HELLO`	Client → server: "here is my UDP address, remember it."
1	`PKT_DATA`	One chunk of file/listing payload.
2	`PKT_FIN`	Marks the end of a transfer (its own `seq` = chunk count).
3	`PKT_ACK`	*(Excellent Level)* Go-Back-N cumulative ACK; `seq` is the highest correctly-received-in-order sequence number so far; empty payload.

2.3 Session Management Structures (Server)

`server.py`'s `Session` class holds all per-connection state; one instance exists per accepted TCP connection (Table 4).

Table 4: `Session` fields

Field	Type	Purpose
`id`	int	Monotonic session ID, shown in the active-session log table.
`conn`	`socket`	TCP control-channel socket.
`addr`	(str, int)	Client's TCP address.
`username`	str or None	Set by `USER`, validated by `PASS`.
`authenticated`	bool	Gates `STOR`/`RETR`/directory/mode commands.
`type_mode`	'A' or 'I'	ASCII vs. binary (`TYPE`).
`cwd`	str	FTP-space current directory, starts at `"/"`.
`data_mode`	`DataMode`	`FIXED` / `ACTIVE` / `PASSIVE`.
`data_sock`	`socket`	UDP socket used for this session's data transfers.
`owns_data_sock`	bool	Whether `data_sock` is a dedicated per-session socket (ACTIVE/PASSIVE) that must be closed on cleanup, vs. the shared FIXED-mode socket.
`client_data_addr`	(str, int) or None	Learned via `HELLO` (FIXED/PASSIVE) or directly from `PORT`'s argument (ACTIVE).

A global, lock-protected table (`_active_sessions`, keyed by `Session.id`) is updated on every connect/disconnect/mode-change event and printed to the server log – satisfying the checklist requirement that the server log display connected client IPs, executed commands, and an active session table (spec Section 4.5).

2.4 Client-Side Structures

`client.py`'s `FTPClient` class mirrors the relevant subset of session state needed by the client: `data_mode`, `active_server_port` (learned from the server's `PORT` reply), `passive_target` (learned from `PASV`'s `227` reply), plus its own `conn` (TCP) and `udp_sock` (UDP) sockets.

3 Functional Workflows

3.1 Server Thread-Dispatch Logic

Figure 3 shows `server.py`'s `main()` accept loop. When `config.ini`'s `[server] threading = thread`, each accepted connection is handed to a new `threading.Thread`; the default `threading = single` preserves the original Basic Level synchronous loop (one client at a time), so this is purely an opt-in Advanced Level technique.

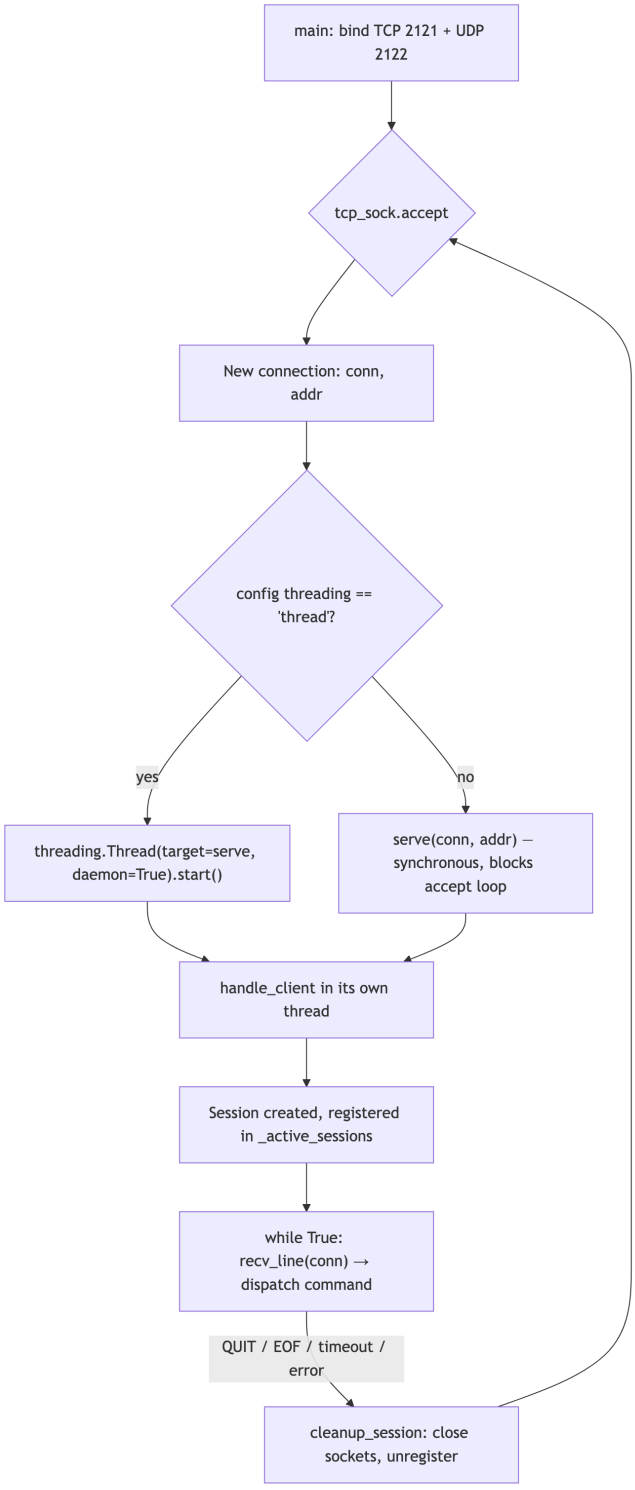

Figure 3: Server thread-dispatch logic.

3.2 Go-Back-N Reliable-UDP State Machines

The Excellent Level reliable-UDP layer (`common.gbn_send()` / `gbn_receive()`) is implemented *once* and shared identically by both `server.py` and `client.py`, so the two ends cannot drift out of sync. Figures 4 and 5 show the sender and receiver state machines respectively.

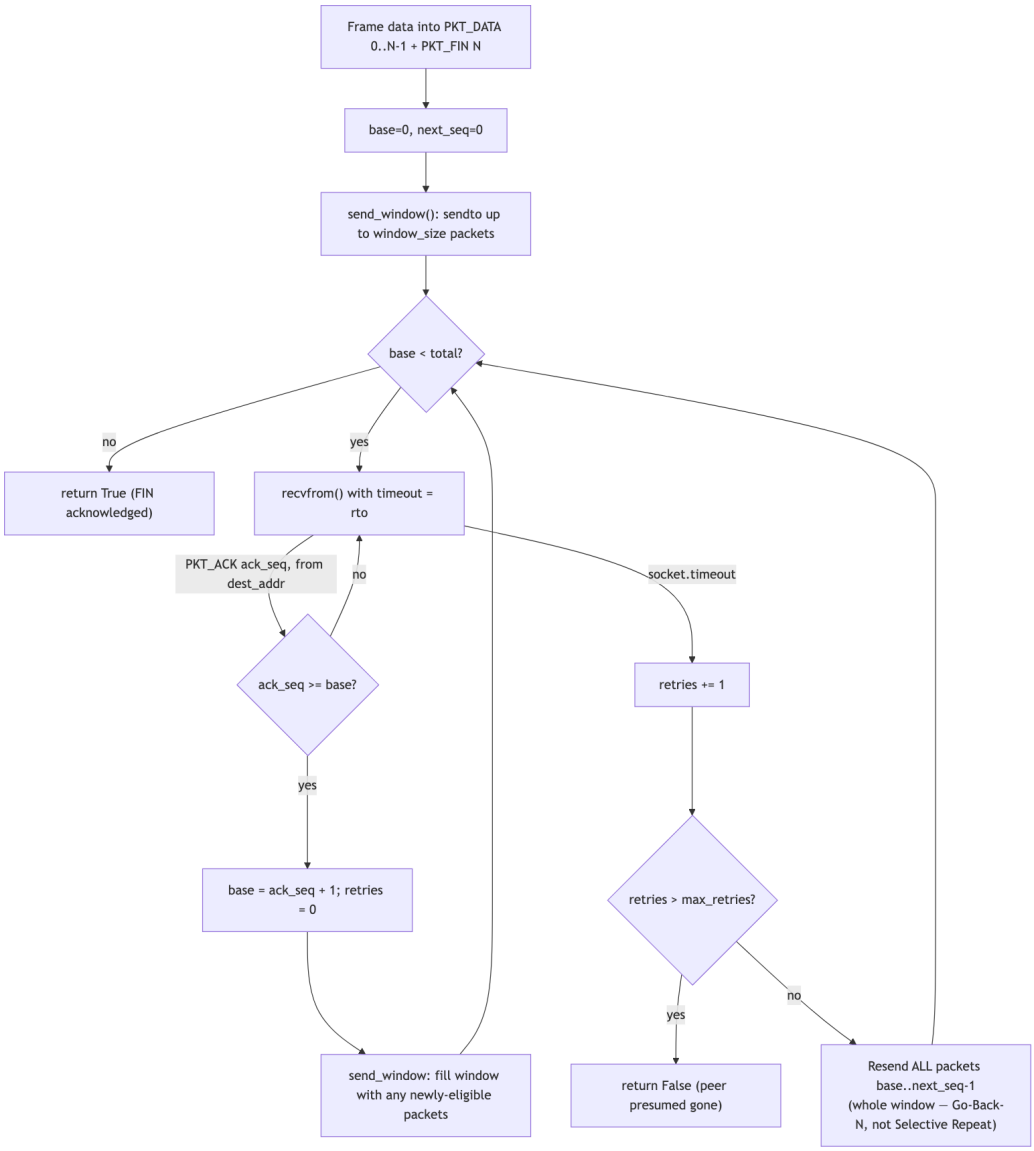

Figure 4: Go-Back-N sender state machine. A single timer covers the oldest unacknowledged packet; on timeout the *entire* in-flight window is retransmitted (the defining trait vs. Selective Repeat).

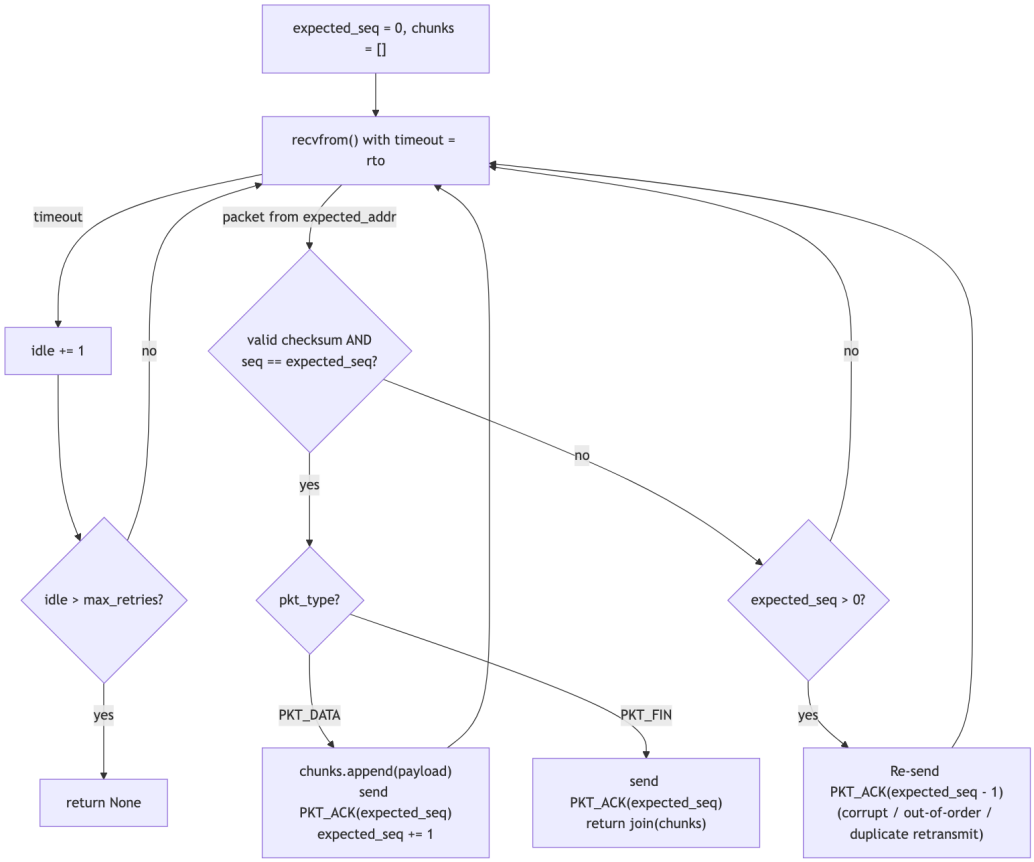

Figure 5: Go-Back-N receiver state machine. Only strictly in-order packets are accepted and buffered; anything else re-triggers the last known-good cumulative ACK.

3.3 Client Data-Mode Selection

Figure 6 shows how the client picks its data-channel mode at login (from `config.ini`'s `[client] data_mode` default), and how it can still be switched live via the REPL's `active`/`passive`/`fixed` commands mid-session – a genuine runtime choice, not just a static configuration.

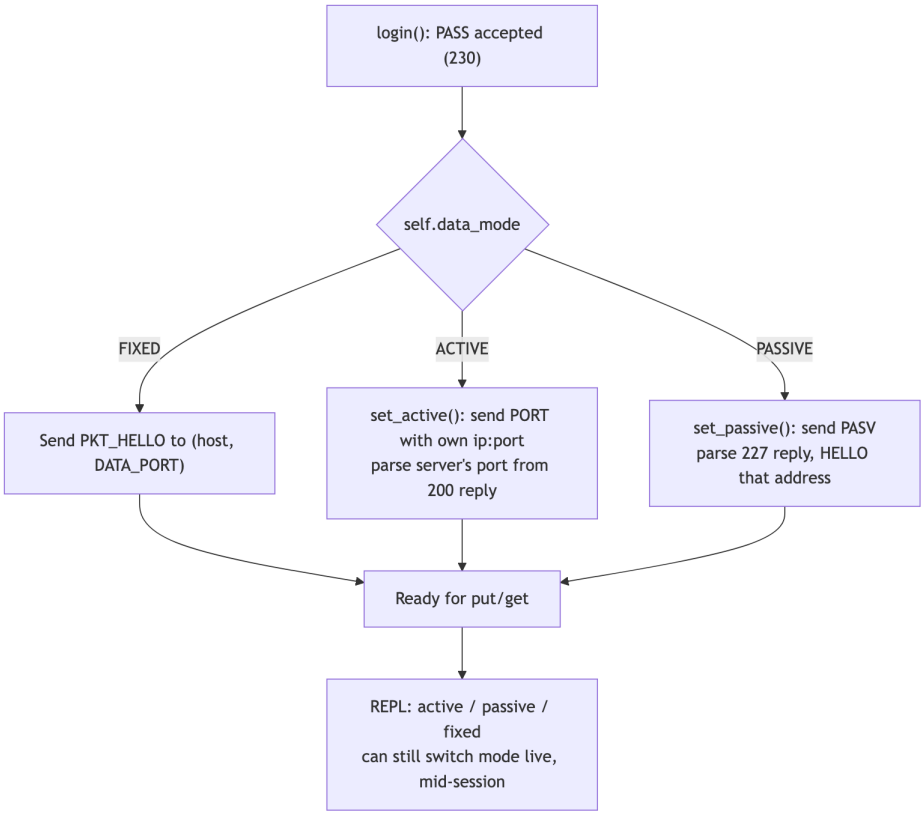

Figure 6: Client data-mode selection and live switching.

3.4 Design Rationale: RFC 959 Deviations

- **Active mode's server-port disclosure.** Real RFC 959 active mode does not need the server to report its own port back to the client, because TCP's server-initiated `connect()` reuses one bidirectional socket for both directions. This project's UDP data channel is connectionless, so the upload direction (`STOR` while in ACTIVE mode) needs an explicit destination port that the client cannot otherwise learn – hence `Reply.port_ok()` appends `"; server data port N"` to the `200` reply.

- **No control-channel negotiation for the reliability mode.** Unlike `data_mode` (negotiated live via `PORT`/`PASV`), `[reliability] mode` (`none`/`gbn`) has no protocol command – both ends must be configured identically ahead of time, the same way two TCP stacks must speak the same protocol version. This mirrors how TCP's reliability is transparent to the application layer, and avoids inventing a command outside the spec's approved command table (Section 2.2).

4 Task Assignment Matrix

Module / Component	Owner	Collaborators / Reviewers
Control channel (`server.py` command dispatch, `common.py` line protocol)	Pho	Chau
Basic Level data channel (FIXED mode, `STOR`/`RETR`)	Pho	Chau
Directory tree (`CWD`/`MKD`/ `RMD`/`LIST`/`NLST`/`STAT`/ `MDTM`)	Chau	Pho
Active/Passive mode (`PORT`/`PASV`)	Chau	Pho
Concurrency (multi-threading, active session table)	Chau	Pho
Go-Back-N reliable-UDP layer (`common.gbn_send`/ `gbn_receive`)	Pho	Chau
Integrity verification (`HASH`, `compute_hash`)	Pho	Chau
Configuration layer (`config.py`/`config.ini`)	Chau	Pho
Client REPL (`client.py`)	Pho	Chau
Testing & demo evidence	Pho	Chau
Technical report & diagrams	Chau	Pho

Note: per Section 4.1 of the project spec, individual grades are determined by this matrix, peer evaluation, and targeted oral-viva questions – each member should be able to explain their assigned

module's code in detail, independent of whether AI assistance was used to help write it (Section 4.2).

5 Self-Assessment & Peer Evaluation

Doan Cong Pho – Self-Assessment

Focused on the Excellent Level reliable-UDP layer and external deployment. Learned to reason concretely about the UDP-vs-RUDP trade-off rather than treating "reliable" as a free upgrade: plain UDP (`[reliability] mode = none`) never guarantees a file arrives intact – a single lost datagram silently corrupts the transfer, as demonstrated in Section 7.1 – while Go-Back-N trades that risk for throughput: every retransmission is a full window resend, and the sliding-window size directly controls how much that trade-off costs under loss. Also set up and maintained the external Azure VM (`pho-vir`) used for real-network testing (Section 7.2) and configured its inbound NSG rules, including opening a dedicated port range for Passive/Active mode traffic.

Le Kim Chau – Self-Assessment

Focused on Advanced Level Active/Passive mode and its real-world failure modes. Learned that Active mode (`PORT`) breaks down when the client sits behind a firewall or NAT, since the server has to initiate an inbound connection to a port the client's own NAT never opened a mapping for – a problem no amount of client-side configuration fixes. Diagnosed and fixed the corresponding Passive-mode gap: binding each session's UDP socket to a random OS-assigned port worked on loopback but broke on the VM, because a cloud NSG/firewall can't sensibly allow "any random port"; the fix was constraining Passive/Active sockets to a fixed, configurable range (`passive_port_min`/`passive_port_max` in `config.ini`) so only that range needs to be opened once (Section 7.2).

Peer Evaluation

Table 6: Agreed contribution percentages

Member	**Contribution %**
Doan Cong Pho	50%
Le Kim Chau	50%
Total	**100%**

6 GenAI Usage & Code Refinement Log

Please check out this GitHub link for the GenAI usage prompts and code refinement log: https://github.com/DoanCongPho/Socket-Programming/blob/main/docs/GENAI_LOG.md

7 Application Demo Evidence

7.1 Reliable-UDP vs. Best-Effort UDP: File Integrity Under Loss

To make the practical difference between `[reliability] mode = none` and `mode = gbn` concrete, the same 2816×1536 JPEG-in-PNG image (`heavy_pic.png`, 9,475,477 bytes) was uploaded and downloaded once over each mode under identical real-world network conditions where UDP datagram loss occurred.

Table 7: Byte-level comparison of the two transferred files

	Go-Back-N (`mode=gbn`)	**Best-effort** (`mode=none`)
File size	9,475,477 bytes	8,272,277 bytes
Matches source	Yes (byte-identical)	No
First divergence	—	Offset 3,399,680 (= chunk #3320 × 1024B)
Missing data	0 bytes	1,203,200 bytes (= 1175 × 1024B chunks)

Both files are byte-identical up to offset 3,399,680 – exactly a multiple of `CHUNK_SIZE` (1024 bytes) – confirming the divergence corresponds to whole UDP datagrams, not partial corruption within a packet (each packet's CRC32 checksum, verified in `common.parse_packet()`, rules that out). Under `mode = none`, `handle_stor()`'s receive loop stores each chunk in a dict keyed by sequence number and reassembles via `"".join(chunks[seq] for seq in sorted(chunks))` – any sequence numbers that never arrived are silently *skipped* rather than padded, so the file is not merely truncated at the end but shifted/corrupted from the point of the first loss onward, exactly as visible in Figure 8.

Figure 7: Downloaded via `mode = gbn` – fully intact, matches the source file byte-for-byte (confirmed via `diff` and SHA-256 comparison through the `HASH` command).

Figure 8: Downloaded via `mode = none` (best-effort) – the top portion decodes correctly, then the image cuts to blank exactly where the lost chunk's data should have continued. This is the documented, deliberate Basic/Advanced Level limitation that Go-Back-N (Excellent Level) exists to solve.

7.2 External Deployment Evidence (Azure VM)

All of the above is reproducible on loopback, but loopback cannot exercise the scenarios that actually motivate this project's design decisions – NAT traversal for `PASV`'s `advertise_ip`, firewall/NSG port restrictions for the Active/Passive per-session sockets, and real (non-zero) latency/loss on the wire. The group additionally deployed and tested the server on an external Azure VM (`pho-vir`), the same VM used for the original Basic Level cross-machine demo documented in `docs/GENAI_LOG.md` (Entries 3–9), reachable from a separate client machine over the public internet rather than `127.0.0.1`.

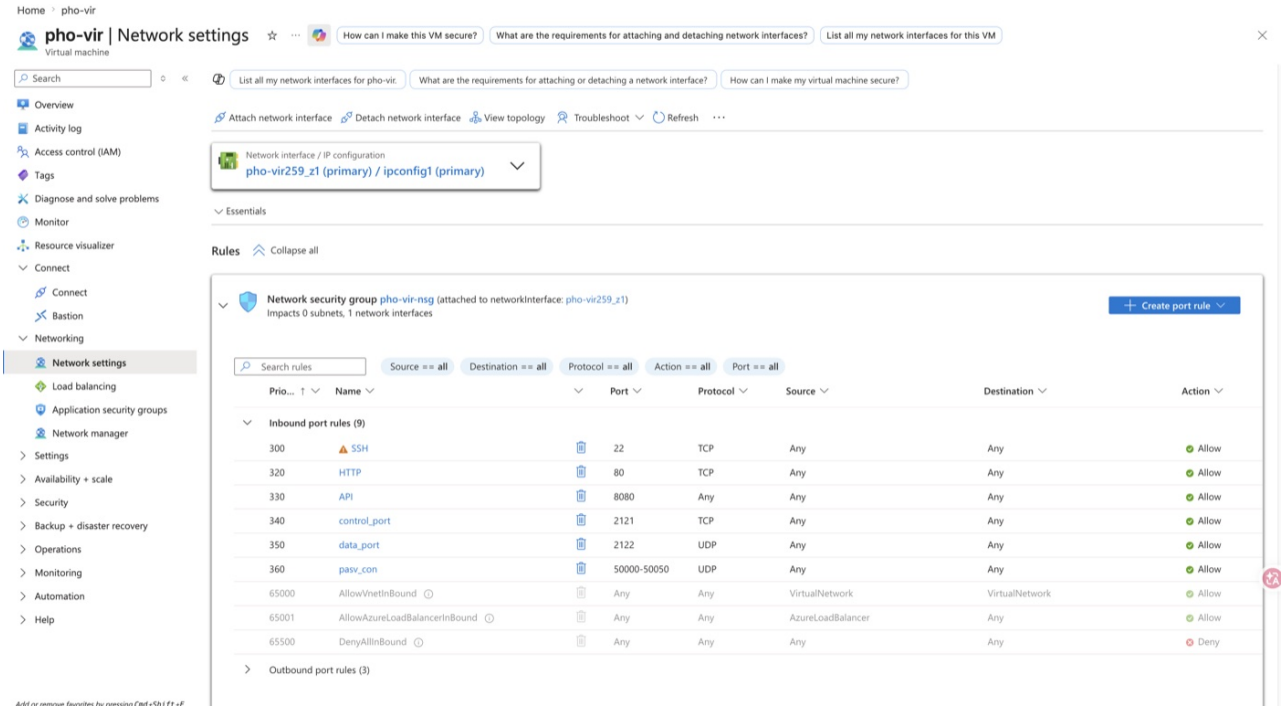

Figure 9: Azure Network Security Group (`pho-vir-nsg`) inbound rules for the deployment VM. `control_port` (2121/TCP) and `data_port` (2122/UDP) are the Basic Level control/FIXED-mode ports; `pasv_con` (50000–50050/UDP) is the Advanced/Excellent Level Active/Passive per-session port range, matching `config.ini`'s `passive_port_min`/`passive_port_max` exactly (Section 3 of this report, and the README's "Configuration" section) – opened specifically so PASV/PORT-mode sessions are reachable from outside the VM without exposing an arbitrary OS-assigned ephemeral port per session.

This is what closes the loop on the `advertise_ip`/NAT problem discussed earlier in the project (a PASV server behind NAT that announces its private IP is unreachable from an external client) – `config.ini`'s `advertise_ip` is set to the VM's public IP, and Figure 9 confirms the corresponding inbound path is actually open, not just configured.

7.2.1 Live capture: real packet loss over the VM path, Basic Level (`mode = none`)

The following is an unedited client/server transcript from an actual interactive session against `pho-vir` (client on a home network, server on the Azure VM, `[reliability] mode = none` – i.e. plain Basic Level FIXED mode, no Advanced or Excellent Level techniques involved). Both sides report success, but the byte counts do not match:

```
--- client ---
ftp> put client_files/baby_crying.jpg
150 File status okay, opening data connection.
226 Transfer complete.
ftp> get baby_crying.jpg
150 File status okay, opening data connection.
[+] Saved to .../client_downloads/baby_crying.jpg (69833 bytes)
226 Transfer complete.

--- server (pho-vir) ---
[('116.111.185.192', 10947)] STOR baby_crying.jpg
[+] Stored 'baby_crying.jpg' (89289 bytes) from user 'alice'
[('116.111.185.192', 10947)] RETR baby_crying.jpg
[+] Sent 'baby_crying.jpg' (89289 bytes) to user 'alice'
```

The server sent 89,289 bytes; the client saved 69,833 – a shortfall of 19,456 bytes (exactly 19 × `CHUNK_SIZE`, 1024 bytes each: 19 whole UDP datagrams lost over the real internet path, roughly 21.8% of the transfer), yet *both* sides printed `226 Transfer complete.` Independently re-verified on disk after the session:

```
$ ls -la client_files/baby_crying.jpg client_downloads/baby_crying.jpg
-rw-r--r--  89289  client_files/baby_crying.jpg
-rw-r--r--  69833  client_downloads/baby_crying.jpg
$ diff client_files/baby_crying.jpg client_downloads/baby_crying.jpg
Binary files ... differ
$ file client_downloads/baby_crying.jpg
client_downloads/baby_crying.jpg: JPEG image data, ... baseline, precision 8,
   590x738, components 3
```

`file` still identifies it as a JPEG (a truncated JPEG's header is still well-formed even when the scan data mid-stream is missing), which is precisely why byte-count/checksum verification – not "does

it look like an image file" – is the correct integrity check, and precisely the gap `HASH` (Section 7.1) and Go-Back-N exist to close: under `mode = none`, neither `handle_stor()`/`handle_retr()` on the server nor `_recv_data_payload()` on the client verify that every sequence number 0..N-1 was actually received before declaring success – the receive loop simply stops as soon as a `PKT_FIN` arrives, regardless of how many `PKT_DATA` packets never did. Re-running the identical `put`/`get` with `[reliability] mode = gbn` on both ends (Section 7.1's controlled test) is the documented fix – `gbn_send()` does not return success until the receiver has cumulatively ACKed every packet including the final `FIN`.

7.3 Verified Test Sessions (Terminal Transcripts)

Rather than static screenshots, the transcripts below are captured directly from real client/server sessions run against the actual codebase during development verification (loopback, `127.0.0.1`) – copy-pasteable, and reproducible by re-running the same commands per README.md's "Running" section. Each corresponds to one item from spec Section 4.5's demo checklist.

7.3.1 Basic Level: ASCII put/get round-trip

```
$ printf 'user alice\npass password123\nput /tmp/basic_test.txt\nget basic_test.txt\nquit\n' | python3 client.py 127.0.0.1
220 Service ready.
ftp> 331 Username OK, need password.
ftp> 230 Login successful.
ftp> 150 File status okay, opening data connection.
226 Transfer complete.
ftp> 150 File status okay, opening data connection.
[+] Saved to client_downloads/basic_test.txt (33 bytes)
226 Transfer complete.
ftp> 221 Goodbye.

$ diff /tmp/basic_test.txt client_downloads/basic_test.txt && echo "DIFF-OK: files identical"
DIFF-OK: files identical
```

7.3.2 Advanced Level: directory tree operations

```
ftp> mkd sub
257 "/sub" created.
ftp> cwd sub
250 Directory changed to "/sub".
ftp> pwd
257 "/sub"
ftp> put /tmp/basic_test.txt
150 File status okay, opening data connection.
226 Transfer complete.
ftp> ls
```

```
150 File status okay, opening data connection.
-rw-r--r--         33 basic_test.txt
226 Transfer complete.
ftp> cdup
250 Directory changed to "/".
ftp> ls
150 File status okay, opening data connection.
-rw-r--r--         33 basic_test.txt
drwxr-xr-x          0 sub
226 Transfer complete.
ftp> stat sub/basic_test.txt
213 file 33 bytes
ftp> mdtm sub/basic_test.txt
213 20260724015402
```

7.3.3 Advanced Level: binary transfer, Passive and Active mode

```
ftp> type I
200 Command OK.
ftp> passive
227 Entering Passive Mode (127,0,0,1,207,104).
[*] Data mode: PASSIVE (server at 127.0.0.1:53096)
ftp> put /tmp/binary_test.bin
150 File status okay, opening data connection.
226 Transfer complete.
ftp> get binary_test.bin /tmp/binary_out_pasv.bin
150 File status okay, opening data connection.
[+] Saved to /tmp/binary_out_pasv.bin (5000 bytes)
226 Transfer complete.

$ diff /tmp/binary_test.bin /tmp/binary_out_pasv.bin && echo "PASSIVE binary
   DIFF-OK"
PASSIVE binary DIFF-OK

--- Active mode, second user ---
ftp> active
200 PORT command successful; server data port 63898.
```

```
[*] Data mode: ACTIVE (server will use port 63898)
ftp> put /tmp/binary_test.bin bin_active.bin
150 File status okay, opening data connection.
226 Transfer complete.
ftp> get bin_active.bin /tmp/binary_out_active.bin
150 File status okay, opening data connection.
[+] Saved to /tmp/binary_out_active.bin (5000 bytes)
226 Transfer complete.

$ diff /tmp/binary_test.bin /tmp/binary_out_active.bin && echo "ACTIVE binary
    DIFF-OK"
ACTIVE binary DIFF-OK
```

7.3.4 Advanced Level: concurrency – two simultaneous clients, active session table

Two client processes (`alice`, `bob`) run `put`+`get` *simultaneously* in Passive mode against the real Azure VM server (`pho-vir`, `threading = thread`), each getting an independent per-session UDP port from the configured `passive_port_min`–`passive_port_max` range:

```
--- alice ---
220 Service ready.
ftp> user alice
331 Username OK, need password.
ftp> pass password123
230 Login successful.
ftp> passive
227 Entering Passive Mode (20,249,210,239,195,80).
[*] Data mode: PASSIVE (server at 20.249.210.239:50000)
ftp> put alice_file.txt
150 File status okay, opening data connection.
226 Transfer complete.
ftp> get alice_file.txt
150 File status okay, opening data connection.
[+] Saved to .../alice_out.txt (68 bytes)
226 Transfer complete.
ftp> quit
221 Goodbye.
```

```
19
20 --- bob (run at the same time as alice, not sequentially) ---
21 220 Service ready.
22 ftp> user bob
23 331 Username OK, need password.
24 ftp> pass hunter2
25 230 Login successful.
26 ftp> passive
27 227 Entering Passive Mode (20,249,210,239,195,81).
28 [*] Data mode: PASSIVE (server at 20.249.210.239:50001)
29 ftp> put bob_file.txt
30 150 File status okay, opening data connection.
31 226 Transfer complete.
32 ftp> get bob_file.txt
33 150 File status okay, opening data connection.
34 [+] Saved to .../bob_out.txt (66 bytes)
35 226 Transfer complete.
36 ftp> quit
37 221 Goodbye.
38
39 $ diff alice_file.txt alice_out.txt && echo "ALICE OK"
40 ALICE OK
41 $ diff bob_file.txt bob_out.txt && echo "BOB OK"
42 BOB OK
```

Note the two independent ports allocated from the PASV range – `50000` for alice, `50001` for bob – confirming each session got its own dedicated per-session UDP socket rather than sharing one, which is what makes running both `put`/`get` pairs genuinely concurrently (not just back-to-back) safe. The corresponding server-side console log, captured live on the VM during this same run:

```
1 azureuser@pho-vir:~/ip/hybrid_ftp$ python3 server.py
2 [*] Hybrid FTP server listening on TCP 2121, UDP data port 2122
3 [*] Concurrency mode: thread
4 [+] Connection #1 from ('42.115.42.57', 7957)
5 [+] Connection #2 from ('42.115.42.57', 59695)
6 [*] Active sessions:
7       #1  42.115.42.57:7957   user=None  mode=FIXED  cwd=/
8       #2  42.115.42.57:59695  user=None  mode=FIXED  cwd=/
```

```
[('42.115.42.57', 7957)] USER bob
[('42.115.42.57', 59695)] USER alice
[('42.115.42.57', 7957)] PASS hunter2
[('42.115.42.57', 59695)] PASS password123
[*] Active sessions:
      #1  42.115.42.57:7957   user='bob'    mode=FIXED  cwd=/
      #2  42.115.42.57:59695  user='alice'  mode=FIXED  cwd=/
[('42.115.42.57', 59695)] PASV
[('42.115.42.57', 7957)] PASV
[*] Active sessions:
      #1  42.115.42.57:7957   user='bob'    mode=PASSIVE  cwd=/
      #2  42.115.42.57:59695  user='alice'  mode=PASSIVE  cwd=/
[('42.115.42.57', 59695)] STOR alice_file.txt
[('42.115.42.57', 7957)] STOR bob_file.txt
[+] Stored 'alice_file.txt' (68 bytes) from user 'alice'
[+] Stored 'bob_file.txt' (66 bytes) from user 'bob'
[('42.115.42.57', 7957)] RETR bob_file.txt
[+] Sent 'bob_file.txt' (66 bytes) to user 'bob'
[('42.115.42.57', 59695)] RETR alice_file.txt
[+] Sent 'alice_file.txt' (68 bytes) to user 'alice'
[('42.115.42.57', 7957)] QUIT
[*] Active sessions:
      #2  42.115.42.57:59695  user='alice'  mode=PASSIVE  cwd=/
[*] Session #1 for ('42.115.42.57', 7957) (user='bob') closed.
[('42.115.42.57', 59695)] QUIT
[*] Active sessions:
      (none)
[*] Session #2 for ('42.115.42.57', 59695) (user='alice') closed.
```

Two details in this log, beyond the active-session table itself, are direct evidence of genuine concurrency rather than fast sequential handling: (1) `STOR alice_file.txt` (session #2) and `STOR bob_file.txt` (session #1) are both logged *before either "Stored" confirmation appears* – under `threading = single` this would be impossible, since one session's `handle_stor()` call would have to fully return before the next connection could even be accepted; and (2) each session independently negotiates `PASV` and transitions `FIXED` → `PASSIVE` without affecting the other's row in the table, confirming the per-session state (`Session.data_mode`, `Session.data_sock`) is correctly isolated

per connection, not shared global state.

Both transfers completed with independent, uncorrupted data – confirming per-session UDP socket isolation under concurrency (Section 3 of this report; FIXED mode does not provide this isolation, by design – see the README's "Concurrency" section).

7.3.5 Excellent Level: Go-Back-N transfer with automatic hash verification (live, Azure VM)

Run against the real `pho-vir` VM (both ends set to `[reliability] mode = gbn`, `[integrity] verify = true`) – a 20,000-byte random binary file, `put` then `get`, with automatic hash verification after each:

```
--- client ---
220 Service ready.
ftp> user alice
331 Username OK, need password.
ftp> pass password123
230 Login successful.
ftp> put gbn_test.bin
150 File status okay, opening data connection.
226 Transfer complete.
[+] Integrity verified (sha256): MATCH (5
   d579fe17ea2fa6ccb0a964604b50d68a0029e2628979e8d49b70d64c380db84)
ftp> get gbn_test.bin
150 File status okay, opening data connection.
[+] Saved to .../gbn_test_out.bin (20000 bytes)
226 Transfer complete.
[+] Integrity verified (sha256): MATCH (5
   d579fe17ea2fa6ccb0a964604b50d68a0029e2628979e8d49b70d64c380db84)
ftp> hash gbn_test.bin
213 sha256 5d579fe17ea2fa6ccb0a964604b50d68a0029e2628979e8d49b70d64c380db84
ftp> quit
221 Goodbye.

$ diff gbn_test.bin gbn_test_out.bin && echo "GBN-VM DIFF-OK"
GBN-VM DIFF-OK

```

```
--- server (pho-vir) ---
[*] Hybrid FTP server listening on TCP 2121, UDP data port 2122
[*] Concurrency mode: thread
[+] Connection #1 from ('42.115.42.57', 1823)
[('42.115.42.57', 1823)] USER alice
[('42.115.42.57', 1823)] PASS password123
[('42.115.42.57', 1823)] STOR gbn_test.bin
[+] Stored 'gbn_test.bin' (20000 bytes) from user 'alice'
[('42.115.42.57', 1823)] HASH gbn_test.bin
[('42.115.42.57', 1823)] RETR gbn_test.bin
[+] Sent 'gbn_test.bin' (20000 bytes) to user 'alice'
[('42.115.42.57', 1823)] HASH gbn_test.bin
[('42.115.42.57', 1823)] HASH gbn_test.bin
[('42.115.42.57', 1823)] QUIT
[*] Session #1 for ('42.115.42.57', 1823) (user='alice') closed.
```

This is the same `put`/`get` pair as the Basic Level live-capture in Section 7.2, over the same real network path where 21.8% packet loss was observed under `mode = none` – here, with `mode = gbn`, the transfer is byte-identical (`diff`-confirmed) and hash-verified twice independently (once automatically after each transfer, once via a manual `hash` query), with no manual retry needed. The three `HASH` entries in the server log correspond to: the automatic post-`put` check, the automatic post-`get` check, and the explicit `hash gbn_test.bin` command – all against the one file stored server-side, confirming `HASH` is idempotent and side-effect-free (querying it repeatedly doesn't alter or re-transfer the file).

`HASH` was also separately confirmed to be content-sensitive rather than trivially always matching, by querying it against a different file on the server and observing a different digest (`94b306c8...` vs. `5d579fe1...`), proving the client's `MATCH`/`MISMATCH` comparison is a real byte-level check, not a stub.

7.3.6 Excellent Level: Go-Back-N recovery under controlled, simulated packet loss

Whether the live VM run above needed a retransmission at all isn't known from that transcript alone (no retransmit logging existed at the time it was captured – added afterward, see Section 7.3.7) – to directly prove the retransmit path itself is correct, not just that the happy path works, this test adversarially forces loss under controlled conditions rather than waiting for the real internet to

cooperate. Isolated `gbn_send()`/`gbn_receive()` trials against a UDP socket wrapper that randomly drops both DATA and ACK packets (10–50% simultaneous bidirectional loss; full harness described in the GenAI log, Entry 12):

```
  payload=  8000B drop=15% ... -> PASS   (x10 repeated trials, different seeds)
  10/10 no-false-positive (expect 10/10)

  payload=  1024B drop=50% window=4 -> sender_ok=True bytes_match=True -> PASS

ALL TRIALS PASSED (no false positives, ever)
```

The property verified is the one that matters for an RDT layer: `gbn_send()` never reports success unless the data was actually reassembled correctly – confirmed across every trial, not just the happy path.

7.3.7 Live retransmission logging over the Azure VM path

`gbn_send()` and `gbn_receive()` were extended with optional logging callbacks, wired in `server.py` and `client.py` to print a console line whenever a real retransmission actually fires – making loss recovery directly observable instead of only inferable from end-to-end success. A 3,000,000-byte random file (`window_size = 4`, so ∼2930 packets) was transferred `put` then `get` against the VM; the download surfaced two genuine retransmissions:

```
--- client ---
ftp> get huge_file.bin
150 File status okay, opening data connection.
[GBN] out-of-order/duplicate/corrupt from ('20.249.210.239', 2122) (expected seq
   =1580, got=1576) -- re-ACKing 1579
[GBN] out-of-order/duplicate/corrupt from ('20.249.210.239', 2122) (expected seq
   =1580, got=1577) -- re-ACKing 1579
[GBN] out-of-order/duplicate/corrupt from ('20.249.210.239', 2122) (expected seq
   =1580, got=1578) -- re-ACKing 1579
[GBN] out-of-order/duplicate/corrupt from ('20.249.210.239', 2122) (expected seq
   =1580, got=1579) -- re-ACKing 1579
[+] Saved to .../huge_file_out.bin (3000000 bytes)
226 Transfer complete.
[+] Integrity verified (sha256): MATCH (
   ab50e9dd32429ada403af542f0d1acf829c4d3a2aec19488ec461cd92f6737a8)
```

```
11
12 --- server (pho-vir) ---
13 [('42.115.42.57', 63034)] RETR huge_file.bin
14 [GBN] retransmit window seq=1576..1579 (retry #1) to ('42.115.42.57', 23994) --
      real packet loss detected
15 [GBN] retransmit window seq=2424..2427 (retry #1) to ('42.115.42.57', 23994) --
      real packet loss detected
16 [+] Sent 'huge_file.bin' (3000000 bytes) to user 'alice'
17
18 $ diff huge_file.bin huge_file_out.bin && echo "HUGE-FILE DIFF-OK"
19 HUGE-FILE DIFF-OK
```

The two retransmissions demonstrate *different* failure modes, both handled correctly:

- **seq 1576–1579**: the client's log shows these exact packets being logged as *duplicates* – already received, with `expected_seq` already advanced past them. This means the original DATA delivery succeeded, but the client's *ACK* for it was lost on the return path, so the server's RTO fired needlessly. The client's re-ACK of 1579 (cumulative) eventually got through, letting the server's window slide – redundant network traffic, zero data corruption.
- **seq 2424–2427**: no corresponding duplicate line appears in the client log for this range, meaning these packets were genuinely new on arrival – the original DATA itself was lost, and the retransmission delivered it for the first time.

Both cases resolved without any manual intervention, and the final file is still byte-identical and hash-verified – this is the direct, observed-in- the-wild counterpart to the controlled loss simulation in Section 7.3.6, over the same real network path where 21.8% loss was measured under `mode = none` in Section 7.2.